

MARAN PMS — Prompt 51: Housekeeping — Lost & Found (Objetos Perdidos)

Archivos involucrados

- `client/src/pages/housekeeping.tsx` — agregar tab y componente `LostFoundTab`
- `server/routes.ts` — nuevos endpoints `/api/lost-found`
- `server/storage.ts` — nuevas funciones
- `shared/schema.ts` — nueva tabla `lost_found_items`

Principio

Simple y operativo. Una tabla, un tab, un formulario. Sin sobre-ingeniería.

1. BASE DE DATOS

Nueva tabla `lost_found_items`

```
CREATE TABLE lost_found_items (  
  id VARCHAR PRIMARY KEY DEFAULT gen_random_uuid(),  
  codigo TEXT NOT NULL UNIQUE,           -- LF-2026-0001 (autogenerado)  
  description TEXT NOT NULL,             -- qué es el objeto  
  category TEXT NOT NULL DEFAULT 'otro', -- 'ropa' | 'electronica' | 'documento'  
  location TEXT NOT NULL,                -- Hab. 305 / Lobby / Restaurant / etc.  
  found_date DATE NOT NULL,  
  found_by TEXT NOT NULL,                -- nombre libre o usuario  
  storage_location TEXT,                 -- dónde está guardado: Depósito, Recepc  
  status TEXT NOT NULL DEFAULT 'en_custodia',  
  -- 'en_custodia' | 'contactado' | 'entregado' | 'descartado'  
  guest_id VARCHAR,                     -- huésped asociado (opcional)  
  reservation_id VARCHAR,               -- reserva asociada (opcional, para busc  
  notes TEXT,                           -- observaciones libres  
  -- Devolución  
  claimed_by TEXT,                       -- nombre de quien retiró  
  claimed_date DATE,
```

```

    delivery_type TEXT,                -- 'retiro_hotel' | 'envio'
    delivered_by TEXT,                 -- quién entregó (operador del hotel)
    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW()
);

```

En `shared/schema.ts` :

```

export type LostFoundStatus = "en_custodia" | "contactado" | "entregado" | "desca
export type LostFoundCategory = "ropa" | "electronica" | "documento" | "accesorio

export const lostFoundItems = pgTable("lost_found_items", {
  id: varchar("id").primaryKey().default(sql`gen_random_uuid()`),
  codigo: text("codigo").notNull().unique(),
  description: text("description").notNull(),
  category: text("category").$type<LostFoundCategory>().notNull().default("otro")
  location: text("location").notNull(),
  foundDate: date("found_date").notNull(),
  foundBy: text("found_by").notNull(),
  storageLocation: text("storage_location"),
  status: text("status").$type<LostFoundStatus>().notNull().default("en_custodia"
  guestId: varchar("guest_id"),
  reservationId: varchar("reservation_id"),
  notes: text("notes"),
  claimedBy: text("claimed_by"),
  claimedDate: date("claimed_date"),
  deliveryType: text("delivery_type"),
  deliveredBy: text("delivered_by"),
  createdAt: timestamp("created_at").defaultNow(),
  updatedAt: timestamp("updated_at").defaultNow(),
});

export const insertLostFoundSchema = createInsertSchema(lostFoundItems).omit({
  id: true, createdAt: true, updatedAt: true
});
export type InsertLostFound = z.infer<typeof insertLostFoundSchema>;
export type LostFoundItem = typeof lostFoundItems.$inferSelect;

```

Generación del código automático (en storage o en el endpoint POST):

```

// Formato: LF-YYYY-NNNN donde NNNN es correlativo del año
async function generateLostFoundCode(): Promise<string> {
  const year = new Date().getFullYear();
  const prefix = `LF-${year}-`;
  const lastItem = await db.select()

```

```

    .from(lostFoundItems)
    .where(like(lostFoundItems.codigo, `${prefix}%`))
    .orderBy(desc(lostFoundItems.createdAt))
    .limit(1);
const lastNum = lastItem[0]
  ? parseInt(lastItem[0].codigo.replace(prefix, ""))
  : 0;
return `${prefix}${String(lastNum + 1).padStart(4, "0")}`;
}

```

2. BACKEND

Endpoints en `server/routes.ts`

```

// Listar todos (con filtros opcionales por status y category)
GET  /api/lost-found
// ?status=en_custodia&category=electronica&search=texto

// Crear
POST /api/lost-found

// Ver detalle
GET  /api/lost-found/:id

// Editar (descripción, notas, storage_location, etc.)
PATCH /api/lost-found/:id

// Cambiar estado (operación principal)
PATCH /api/lost-found/:id/status
// body: { status, claimedBy?, claimedDate?, deliveryType?, deliveredBy?, notes?

// Buscar reserva/huésped por habitación (para asociar al objeto)
GET  /api/lost-found/lookup-room/:roomNumber
// Retorna: última reserva de esa habitación y datos del huésped

```

Implementación del GET con filtros:

```

app.get("/api/lost-found", requireAuth, async (req, res) => {
  try {
    const { status, category, search } = req.query;
    let query = db.select().from(lostFoundItems);

    const conditions = [];

```

```

if (status) conditions.push(eq(foundItems.status, status as string));
if (category) conditions.push(eq(foundItems.category, category as string));
if (search) {
  const q = `%${search}%`;
  conditions.push(or(
    ilike(foundItems.description, q),
    ilike(foundItems.location, q),
    ilike(foundItems.codigo, q),
    ilike(foundItems.foundBy, q),
  ));
}

const items = conditions.length > 0
  ? await db.select().from(foundItems).where(and(...conditions)).orderBy(
    : await db.select().from(foundItems).orderBy(desc(foundItems.create

res.json(items);
} catch (error) {
  res.status(500).json({ error: "Error fetching lost and found items" });
}
});

```

Endpoint lookup-room — útil para asociar el objeto a un huésped sin buscarlo manualmente:

```

app.get("/api/lost-found/lookup-room/:roomNumber", requireAuth, async (req, res)
  try {
    const { roomNumber } = req.params;
    const room = await storage.getRoomByNumber(roomNumber);
    if (!room) return res.status(404).json({ error: "Habitación no encontrada" });

    // Última reserva de esa habitación (checkedin o checked_out reciente)
    const recentReservation = await db.select()
      .from(reservations)
      .where(eq(reservations.roomId, room.id))
      .orderBy(desc(reservations.checkOutDate))
      .limit(1);

    if (!recentReservation[0]) return res.json({ room, reservation: null, guest:

    const guest = recentReservation[0].guestId
      ? await storage.getGuest(recentReservation[0].guestId)
      : null;

    res.json({
      room,

```

```

        reservation: recentReservation[0],
        guest
    });
} catch (error) {
    res.status(500).json({ error: "Error looking up room" });
}
});

```

3. FRONTEND

3.1 Agregar tab en Housekeeping

En `client/src/pages/housekeeping.tsx`, envolver la vista actual en un `<Tabs>` y agregar el tab de Lost & Found:

```

// Reemplazar el return actual por:
return (
    <div className="flex flex-col gap-4 p-6">
        {/* Header existente - no cambiar */}
        <div className="flex items-center justify-between">
            <div className="flex items-center gap-3">
                <ClipboardCheck className="h-6 w-6 text-muted-foreground" />
                <h1 className="text-2xl font-bold">Housekeeping</h1>
            </div>
        </div>

        <Tabs defaultValue="rooms">
            <TabsList>
                <TabsTrigger value="rooms">Habitaciones</TabsTrigger>
                <TabsTrigger value="lost-found" data-testid="tab-lost-found">
                    <Package className="h-4 w-4 mr-1" />
                    Objetos Perdidos
                    {lostFoundCount > 0 && (
                        <Badge className="ml-1 h-5 w-5 p-0 text-xs flex items-center justify-
                            {lostFoundCount}
                        </Badge>
                    )}
                </TabsTrigger>
            </TabsList>

            <TabsContent value="rooms">
                {/* Todo el contenido actual de Housekeeping sin cambios */}
            </TabsContent>

```

```

        <TabsContent value="lost-found">
          <LostFoundTab />
        </TabsContent>
      </Tabs>
    </div>
  );

```

El badge en el tab muestra la cantidad de objetos con status "en_custodia" o "contactado" (los activos). Obtenerlo con:

```

const { data: lostFoundActive = [] } = useQuery<LostFoundItem[]>({
  queryKey: ["/api/lost-found", { status: "en_custodia" }],
  queryFn: () => fetch("/api/lost-found?status=en_custodia").then(r => r.json()),
});
const lostFoundCount = lostFoundActive.length;

```

3.2 Componente `LostFoundTab`

```

function LostFoundTab() {
  const { toast } = useToast();
  const [showForm, setShowForm] = useState(false);
  const [editingItem, setEditingItem] = useState<LostFoundItem | null>(null);
  const [statusFilter, setStatusFilter] = useState<string>("activos");
  const [searchText, setSearchText] = useState("");
  const [showDeliveryDialog, setShowDeliveryDialog] = useState(false);
  const [deliveringItem, setDeliveringItem] = useState<LostFoundItem | null>(null);

  const { data: items = [], isLoading } = useQuery<LostFoundItem[]>({
    queryKey: ["/api/lost-found", statusFilter, searchText],
    queryFn: () => {
      const params = new URLSearchParams();
      if (statusFilter === "activos") {
        // No filtrar por status - traer todos menos entregados/descartados
        // (filtrar en frontend)
      } else if (statusFilter !== "todos") {
        params.set("status", statusFilter);
      }
      if (searchText) params.set("search", searchText);
      return fetch(`/api/lost-found?${params}`).then(r => r.json());
    },
  });

  const visibleItems = statusFilter === "activos"
    ? items.filter(i => i.status === "en_custodia" || i.status === "contactado")
    : items;

```

```

// Mutations
const createMutation = useMutation({
  mutationFn: (data: Partial<InsertLostFound>) =>
    apiRequest("POST", "/api/lost-found", data).then(r => r.json()),
  onSuccess: () => {
    queryClient.invalidateQueries({ queryKey: ["/api/lost-found"] });
    setShowForm(false);
    toast({ title: "Objeto registrado" });
  },
});

const updateStatusMutation = useMutation({
  mutationFn: ({ id, ...data }: { id: string; status: string; claimedBy?: string }) =>
    apiRequest("PATCH", `/api/lost-found/${id}/status`, data).then(r => r.json()),
  onSuccess: () => {
    queryClient.invalidateQueries({ queryKey: ["/api/lost-found"] });
    setShowDeliveryDialog(false);
    toast({ title: "Estado actualizado" });
  },
});

return (
  <div className="space-y-4 mt-4">
    {/* Header con botón registrar y filtros */}
    <div className="flex items-center justify-between gap-4">
      <div className="flex items-center gap-2 flex-1">
        <div className="relative flex-1 max-w-sm">
          <Search className="absolute left-3 top-1/2 -translate-y-1/2 h-4 w-4 t
          <Input
            placeholder="Buscar por descripción, habitación, código..."
            value={searchText}
            onChange={(e) => setSearchText(e.target.value)}
            className="pl-9"
          />
        </div>
      </div>
      <Select value={statusFilter} onValueChange={setStatusFilter}>
        <SelectTrigger className="w-36">
          <SelectValue />
        </SelectTrigger>
        <SelectContent>
          <SelectItem value="activos">Activos</SelectItem>
          <SelectItem value="en_custodia">En custodia</SelectItem>
          <SelectItem value="contactado">Contactado</SelectItem>
          <SelectItem value="entregado">Entregados</SelectItem>
          <SelectItem value="descartado">Descartados</SelectItem>
          <SelectItem value="todos">Todos</SelectItem>
        </SelectContent>
      </Select>
    </div>
  </div>
)

```

```

        </SelectContent>
    </Select>
</div>
<Button onClick={() => { setEditingItem(null); setShowForm(true); }} data
    <Plus className="h-4 w-4 mr-1" /> Registrar objeto
</Button>
</div>

{/* Lista de objetos */}
{isLoading ? (
    <div className="space-y-2">[...Array(3)].map((_, i) => <Skeleton key={i}
) : visibleItems.length === 0 ? (
    <div className="text-center py-12 text-muted-foreground">
        <Package className="h-10 w-10 mx-auto mb-3 opacity-30" />
        <p className="text-sm">No hay objetos registrados</p>
    </div>
) : (
    <div className="space-y-2">
        {visibleItems.map(item => (
            <LostFoundCard
                key={item.id}
                item={item}
                onEdit={() => { setEditingItem(item); setShowForm(true); }}
                onDeliver={() => { setDeliveringItem(item); setShowDeliveryDialog(t
                onChange={status => updateStatusMutation.mutate({ id: item.
            </div>
        )}}
    </div>
)}

{/* Dialog de formulario (crear/editar) */}
{showForm && (
    <LostFoundForm
        item={editingItem}
        open={showForm}
        onOpenChange={setShowForm}
        onSubmit={data => {
            if (editingItem) {
                // update
            } else {
                createMutation.mutate(data);
            }
        }}
    </div>
)}

{/* Dialog de entrega */}

```

```

    {showDeliveryDialog && deliveringItem && (
      <DeliveryDialog
        item={deliveringItem}
        open={showDeliveryDialog}
        onOpenChange={setShowDeliveryDialog}
        onConfirm={(data) => updateStatusMutation.mutate({ id: deliveringItem.i
      />
    ))
  </div>
);
}

```

3.3 Card de objeto (`LostFoundCard`)

```

function LostFoundCard({ item, onEdit, onDeliver, onStatusChange }) {
  const statusConfig = {
    en_custodia: { label: "En custodia", className: "bg-blue-100 text-blue-700" }
    contactado: { label: "Contactado", className: "bg-amber-100 text-amber-700" }
    entregado: { label: "Entregado", className: "bg-green-100 text-green-700" }
    descartado: { label: "Descartado", className: "bg-gray-100 text-gray-500" }
  };

  const categoryIcons = {
    ropa: "👕", electronica: "📱", documento: "📄", accesorio: "👜", otro: "📦"
  };

  const daysInCustody = Math.floor(
    (Date.now() - new Date(item.foundDate).getTime()) / 86400000
  );

  return (
    <div className="flex items-start justify-between p-4 border rounded-lg hover:
      data-testid={`lost-found-card-${item.id}`}>
      <div className="flex items-start gap-3">
        <span className="text-2xl">{categoryIcons[item.category]} || "📦"</span>
      </div>
      <div className="flex items-center gap-2">
        <span className="font-medium">{item.description}</span>
        <Badge className={statusConfig[item.status].className}>
          {statusConfig[item.status].label}
        </Badge>
        {daysInCustody > 30 && item.status === "en_custodia" && (
          <Badge variant="destructive" className="text-xs">+30 días</Badge>
        )}
      </div>
      <div className="text-sm text-muted-foreground mt-0.5">
        <span className="font-mono">{item.codigo}</span>
        {" · "}
      </div>
    </div>
  );
}

```

```

        {item.location}
        {" · "}
        {new Date(item.foundDate + "T12:00:00").toLocaleDateString("es-AR")}
        {" · "}
        Encontrado por: {item.foundBy}
    </div>
    {item.storageLocation && (
        <p className="text-xs text-muted-foreground">Guardado en: {item.stora
    )}
    {item.notes && (
        <p className="text-xs text-muted-foreground italic mt-0.5">{item.note
    )}
    </div>
</div>
<div className="flex items-center gap-1 ml-4 shrink-0">
    {item.status === "en_custodia" && (
        <Button size="sm" variant="outline"
            onClick={() => onStatusChange("contactado")}
            data-testid={`button-contacted-${item.id}`}>
            Contactado
        </Button>
    )}
    {(item.status === "en_custodia" || item.status === "contactado") && (
        <Button size="sm"
            onClick={onDeliver}
            data-testid={`button-deliver-${item.id}`}>
            Entregar
        </Button>
    )}
    <Button size="icon" variant="ghost" onClick={onEdit}>
        <Pencil className="h-4 w-4" />
    </Button>
</div>
</div>
);
}

```

3.4 Formulario de registro (`LostFoundForm`)

Campos del formulario:

```

// Campos obligatorios:
- description: Input (qué es el objeto)
- category: Select (Ropa / Electrónica / Documento / Accesorio / Otro)
- location: Input con placeholder "Ej: Hab. 305, Lobby, Restaurant..."
  + botón "Buscar huésped" que llama a /api/lost-found/lookup-room/[número]

```

```

- found_date: Input type="date" (default hoy)
- found_by: Input (default: usuario logueado)

// Campos opcionales:
- storageLocation: Input (dónde se guarda: Depósito, Recepción...)
- notes: Textarea

// Resultado del lookup de habitación (si se encontró):
// Mostrar: "Último huésped: [nombre] - Check-out: [fecha]"
// Con botón para asociar a esa reserva/huésped

```

3.5 Dialog de entrega (`DeliveryDialog`)

```

// Campos:
- claimedBy: Input (nombre de quien retira)
- claimedDate: Input type="date" (default hoy)
- deliveryType: Select ("Retiro en hotel" / "Envío")
- deliveredBy: Input (operador que entrega)
- notes: Textarea (opcional)

// Al confirmar: status → "entregado"

```

4. REGLAS DE NEGOCIO

- **No eliminar registros** — solo cambiar estado. No exponer botón DELETE en la UI.
 - Los objetos con más de **30 días** en custodia muestran badge de advertencia rojo.
 - El código `LF-YYYY-NNNN` se genera automáticamente en el backend — no editable.
 - Al registrar un objeto encontrado en una habitación, el botón "Buscar huésped" consulta `/api/lost-found/lookup-room/:roomNumber` y sugiere el huésped automáticamente, pero la asociación es opcional.
 - El tab en housekeeping muestra un badge con la cantidad de objetos activos (`en_custodia + contactado`).
-

NOTAS TÉCNICAS

- Package `icon` de `lucide-react` para el tab y el estado vacío.

- `getRoomByNumber` **puede no existir en storage — si no existe, agregar:**

```
async getRoomByNumber(roomNumber: string): Promise<Room | undefined> {  
  const [room] = await db.select().from(rooms).where(eq(rooms.roomNumber, roomNumber))  
  return room;  
}
```

- **La tabla `lost_found_items` no referencia `rooms` con FK para mantenerlo flexible (puede ser área, no solo habitación).**
- `like` e `ilike` de Drizzle ya están importados en el archivo si se usan en otros endpoints.
- **Toda la UI del tab de habitaciones existente va dentro de `<TabsContent value="rooms">` sin ningún cambio.**